📝 APUNTES DE PREPARACIÓN: CONTROL CHECK 1 - DP1 (SPRING BOOT + REACT)

Versión Completa y Unificada - La "Chuleta Maestra" para cualquier examen

---

🛑 REGLAS DE ORO DURANTE EL EXAMEN

1. **NUNCA modifiques firmas de métodos ni nombres de clases dados.** Los tests del profesor fallarán.
2. **NUNCA modifiques los tests unitarios proporcionados.**
3. **Si el proyecto no compila, la nota es 0.** Comprueba siempre que no hay errores de sintaxis.
4. Es normal que `mvnw test` falle al principio. Resuelve los tests uno a uno.

---

1. ENTIDADES Y MAPEO BÁSICO (JPA)

1.1. Estructura Básica

```java
@Entity
@Table(name = "custom_table_name") // Úsalo si el nombre en BD es diferente al de la clase
public class MiClase extends BaseEntity { // BaseEntity ya suele incluir el @Id

    // Si NO hereda de BaseEntity:
    // @Id
    // @GeneratedValue(strategy = GenerationType.IDENTITY)
    // Integer id;
}
```

1.2. El Diccionario de Validaciones (¿Cuándo usar qué?)

Textos / Cadenas de caracteres (`String`)

- `@NotNull`: NUNCA lo uses solo en Strings. Permite cadenas vacías (`""`).
- `@NotBlank`: **El rey de los Strings.** Úsalo cuando el texto es obligatorio. Falla si es `null`, si está vacío o si solo son espacios en blanco.

- `@NotEmpty`: Falla si es `null` o vacío, pero permite espacios. Mejor usa `@NotBlank`.
- `@Size(min = 3, max = 50)`: Para limitar la longitud de la cadena.
- `@Email`: Valida que el formato sea un email válido (ej: `test@test.com`).
- `@URL`: Valida que sea una URL válida (ej: `http://...`).
- `@Pattern(regexp = "^[0-9]{9}$")`: Usa expresiones regulares. Úsalo para DNIs, teléfonos, matrículas, etc.

Números (`Integer`, `Double`, etc.)

- `@NotNull`: Obligatorio en números si no pueden estar vacíos (aquí no sirve `@NotBlank`).
- `@Min(0)`: Valor mínimo (ej: para precios, edades, victorias no negativas).
- `@Max(100)`: Valor máximo (ej: porcentajes, límites de capacidad).
- `@Digits(integer = 3, fraction = 2)`: Define cuántos enteros y decimales se permiten.

Fechas (`LocalDate`, `LocalDateTime`)

- `@NotNull`: Obligatorio.
- `@Past` / `@PastOrPresent`: La fecha debe ser en el pasado. Úsalo para fechas de nacimiento, fechas de creación.
- `@Future` / `@FutureOrPresent`: La fecha debe ser en el futuro. Úsalo para fechas de caducidad, reservas de citas.

Colecciones (`List`, `Set`)

- `@NotEmpty`: La lista no puede estar vacía (debe tener al menos 1 elemento).
- `@Size(min = 2, max = 5)`: La lista debe tener un número específico de elementos.

1.3. Enumerados y Fechas

```java
// OBLIGATORIO: Si no pones EnumType.STRING, guarda un número en BD (0, 1...) y fallará si añades nuevos enums.
@NotNull
@Enumerated(EnumType.STRING)
private Status status;

@NotNull
@DateTimeFormat(pattern = "yyyy/MM/dd") // Opcional, para forzar formatos de entrada
private LocalDate startDate;
```

1.4. Atributos Calculados (El truco del `@Transient`)

Si en el diagrama UML ves un atributo con un `/` delante (ej: `/ratio`), NO es una columna de la base de datos.

```java
@Transient // Hibernate ignora esto, no busca la columna en la BD
public Double getRatio() {
    return (double) victorias / partidas;
}
```

---

2. RELACIONES JPA: ¿CUÁNDO USAR CUÁL?

Para decidir, lee la relación en ambos sentidos (Ej: Un "Coche" tiene muchas "Ruedas". Una "Rueda" pertenece a un "Coche").

2.1. `@ManyToOne` (Muchos a Uno) - La más común

- **Cuándo usar:** Un elemento hijo pertenece a un solo padre.
    - *Ejemplo:* Una Mascota tiene un Dueño. Un Dueño tiene muchas Mascotas.
- **Dónde se pone:** En la clase Mascota (el hijo).

```java
@ManyToOne(optional = false) // optional = false hace que el dueño sea obligatorio (NOT NULL)
@JoinColumn(name = "owner_id") // Nombre de la columna que será Clave Foránea en la tabla Mascota
private Owner owner;
```

2.2. `@OneToMany` (Uno a Muchos)

- **Cuándo usar:** Quieres ver los hijos desde el padre.
- **OJO:** Suele ir acompañada de un `@ManyToOne` en la otra clase (Relación Bidireccional).

```java
// En la clase Owner (Padre)
@OneToMany(mappedBy = "owner", cascade = CascadeType.ALL)
// mappedBy = "owner" -> Le dice a JPA: "La relación ya está definida en el atributo 'owner' de la clase Mascota. No crees tablas intermedias extrañas".
private List<Pet> pets;
```

Si es **Unidireccional** (Solo existe en el padre):

```java
@OneToMany
@JoinColumn(name = "parent_id") // OBLIGATORIO si es unidireccional para evitar tablas intermedias.
private List<Hijo> hijos;
```

2.3. `@ManyToMany` (Muchos a Muchos)

- **Cuándo usar:** Varios elementos de A pueden estar en varios elementos de B.
    - *Ejemplo:* Un Estudiante se matricula en muchos Cursos. Un Curso tiene muchos Estudiantes.

```java
// En la clase dominante (ej. Curso)
@ManyToMany
@JoinTable(
    name = "curso_estudiantes", // Nombre de la tabla intermedia
    joinColumns = @JoinColumn(name = "curso_id"),
    inverseJoinColumns = @JoinColumn(name = "estudiante_id")
)
// Añade @Size(min=1) si el UML exige que un curso tenga al menos 1 estudiante.
private List<Estudiante> estudiantes;
```

2.4. `@OneToOne` (Uno a Uno)

- **Cuándo usar:** Una entidad pertenece única y exclusivamente a otra.
    - *Ejemplo:* Un User tiene un Profile.

```java
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "profile_id", referencedColumnName = "id")
private Profile profile;
```

---

3. SCRIPTS SQL (`data.sql`)

Reglas para Inserts

1. **ORDEN:** Inserta primero las tablas independientes (padres), luego las dependientes (hijos, con claves foráneas).
    - *Ej:* 1º `Users`, 2º `Owners` (usan `user_id`), 3º `Pets` (usan `owner_id`).

2. **Cadenas:** Comillas simples `'Texto'`. Para apóstrofes, dobla la comilla: `'I''m ready'`.
3. **Fechas y Tiempos:**
    - `LocalDate`: `'2023-12-31'`
    - `LocalDateTime`: `'2023-12-31 15:30:00'`
4. **Booleanos:** En H2 suelen funcionar `TRUE`/`FALSE` o `1`/`0`.
5. **Tablas Intermedias (`@ManyToMany`):** Tienes que hacer el `INSERT` a mano en la tabla que se crea.

```sql
INSERT INTO curso_estudiantes (curso_id, estudiante_id) VALUES (1, 5);
```

---

4. REPOSITORIOS (`@Repository`) - SUPER VITAMINADO

El repositorio se encarga exclusivamente de hablar con la Base de Datos. NUNCA pongas lógica de negocio aquí.

4.1. El Poder del "Derived Query" (Magia de Spring)

Si usas los prefijos y palabras clave correctas, no necesitas escribir SQL.

```java
@Repository
public interface PartidaRepository extends CrudRepository<Partida, Integer> {

    // Búsquedas Básicas
    List<Partida> findByStatus(Status status);
    Optional<Partida> findByCode(String code); // Usa Optional si esperas 0 o 1 resultado

    // Múltiples Condiciones (AND / OR)
    List<Partida> findByStatusAndCreatorId(Status status, Integer creatorId);
    List<Partida> findByStartDateAfterAndEndDateBefore(LocalDateTime start, LocalDateTime end);

    // Cadenas de Texto (LIKE)
    List<Partida> findByNameContainingIgnoreCase(String name); // LIKE %name% ignorando mayúsculas
    List<Partida> findByNameStartingWith(String prefix);

    // Colecciones y Nulos
    List<Partida> findByWinnerIsNull();
```

```
    List<Partida> findByStatusIn(List<Status> statuses); // Ej: pasas
[IN_PROGRESS, FINISHED]

    // Navegación por Relaciones Mágica
    List<Partida> findByCreatorUsername(String username);

    // Ordenación y Contadores
    List<Partida> findByStatusOrderByStartDateDesc(Status status);
    boolean existsByCode(String code);
    int countByStatus(Status status);
}
```

4.2. Consultas Complejas con `@Query` (JPQL)

Usa JPQL cuando las derived queries sean muy largas o necesites hacer `JOIN`, agrupar o contar. **Recuerda: Usa nombres de CLASES y ATRIBUTOS Java, no tablas SQL.**

Java

```
@Repository
public interface JugadorRepository extends CrudRepository<Jugador, Integer> {

    // 1. JOIN BÁSICO (Buscar jugadores en una partida específica)
    @Query("SELECT j FROM Jugador j JOIN j.partidas p WHERE p.id = :partidaId")
    List<Jugador> findJugadoresEnPartida(@Param("partidaId") Integer partidaId);

    // 2. CONTAR ELEMENTOS DE UNA RELACIÓN (Ej: Número de partidas jugadas por un
jugador)
    @Query("SELECT COUNT(p) FROM Partida p JOIN p.jugadores j WHERE j.id =
:jugadorId")
    Integer countPartidasByJugador(@Param("jugadorId") Integer jugadorId);

    // 3. ACTUALIZACIÓN MASIVA (Requiere @Modifying)
    @Modifying
    @Query("UPDATE Jugador j SET j.estado = 'INACTIVO' WHERE j.ultimaConexion <
:fechaLimite")
    void desactivarJugadoresInactivos(@Param("fechaLimite") LocalDate
fechaLimite);

    // 4. CONSULTA DE ESTADÍSTICAS / RANKING (Agrupación y Orden)
    // Devuelve un Object[] porque no es una entidad completa, son columnas
sueltas.
    @Query("SELECT j.username, COUNT(p) FROM Jugador j JOIN j.partidasGanadas p
GROUP BY j.username ORDER BY COUNT(p) DESC")
    List<Object[]> obtenerRankingVictorias();

    // 5. NATIVE QUERY (SQL Puro - Úsalo solo como último recurso)
```

```
    @Query(value = "SELECT * FROM jugadores WHERE username LIKE '%test%'",
nativeQuery = true)
    List<Jugador> findTestUsersNative();
}
```

---

5. LÓGICA DE NEGOCIO (`@Service`) - PATRONES COMPLETOS

El servicio es el "Cerebro". Aquí validas reglas del juego o negocio, calculas cosas y decides si llamar al repositorio o lanzar una excepción.

Java

```java
@Service
public class PartidaService {

    @Autowired private PartidaRepository partidaRepo;
    @Autowired private JugadorService jugadorService; // Puedes inyectar otros
servicios

    // Búsquedas de solo lectura (Mejora el rendimiento)
    @Transactional(readOnly = true)
    public List<Partida> findAll() {
        return (List<Partida>) partidaRepo.findAll();
    }

    @Transactional(readOnly = true)
    public Partida findById(Integer id) {
        return partidaRepo.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Partida no
encontrada con ID: " + id));
    }

    // ========================================
    // CREACIÓN (Reglas de negocio típicas)
    // ========================================
    @Transactional(rollbackFor = Exception.class)
    public Partida create(Partida nuevaPartida, Integer creadorId) throws
BusinessException {
        Jugador creador = jugadorService.findById(creadorId);

        // REGLA 1: Un jugador no puede crear partida si ya tiene una en curso
        boolean tieneEnCurso = partidaRepo.existsByCreatorIdAndStatus(creadorId,
Status.IN_PROGRESS);
        if (tieneEnCurso) {
            throw new BusinessException("Ya tienes una partida en curso.");
```

```
        }

        // REGLA 2: Validar fechas lógicas
        if (nuevaPartida.getStartDate() != null && nuevaPartida.getEndDate() !=
null) {
            if (nuevaPartida.getStartDate().isAfter(nuevaPartida.getEndDate())) {
                throw new BusinessException("La fecha de inicio debe ser anterior
al fin.");
            }
        }

        // REGLA 3: Valores por defecto
        nuevaPartida.setStatus(Status.WAITING);
        nuevaPartida.setStartDate(LocalDateTime.now());
        nuevaPartida.setCreator(creador);

        // REGLA 4: Validar que el máximo de jugadores sea lógico
        if (nuevaPartida.getMaxPlayers() < 2 || nuevaPartida.getMaxPlayers() > 8)
{
            throw new BusinessException("El número de jugadores debe estar entre 2
y 8.");
        }

        return partidaRepo.save(nuevaPartida);
    }

    // ========================================
    // ACTUALIZACIÓN (Cuidado con sobrescribir)
    // ========================================
    @Transactional
    public Partida update(Integer partidaId, Partida datosActualizados) throws
BusinessException {
        Partida partidaExistente = findById(partidaId);

        // REGLA: No se puede editar una partida que ya ha terminado
        if (partidaExistente.getStatus() == Status.FINISHED) {
            throw new BusinessException("No puedes modificar una partida
finalizada.");
        }

        // Actualizamos SOLO los campos permitidos
        partidaExistente.setName(datosActualizados.getName());
        partidaExistente.setMaxPlayers(datosActualizados.getMaxPlayers());

        // El save() en una entidad que ya existe hace un UPDATE automático.
        return partidaRepo.save(partidaExistente);
    }
```

```
    // ========================================
    // BORRADO (Controlar restricciones)
    // ========================================
    @Transactional
    public void delete(Integer partidaId) throws BusinessException {
        Partida partida = findById(partidaId);

        // REGLA: Solo se pueden borrar partidas en estado WAITING
        if (partida.getStatus() != Status.WAITING) {
            throw new BusinessException("No se puede borrar una partida que ya ha
empezado.");
        }

        partidaRepo.delete(partida);
    }
}
```

---

6. CONTROLADORES API REST (`@RestController`) - CRUD COMPLETO

El controlador debe ser lo más "tonto" posible. Su trabajo es: Recibir HTTP -> Llamar al Service -> Devolver HTTP.

Diccionario de Códigos HTTP más usados:

- `200 OK`: Para lecturas (GET) o actualizaciones (PUT) exitosas.
- `201 CREATED`: Para creaciones (POST) exitosas.
- `400 BAD REQUEST`: Validación fallida o regla de negocio rota.
- `403 FORBIDDEN`: Usuario sin permisos.
- `404 NOT FOUND`: La entidad (el ID) no existe.

```
Java
@RestController
@RequestMapping("/api/v1/partidas")
public class PartidaController {

    @Autowired
    private PartidaService partidaService;
    @Autowired
    private UserService userService; // Para sacar el usuario logueado

    // 1. GET ALL (Devolver listas)
```

```
    @GetMapping
    public ResponseEntity<List<Partida>> getAllPartidas(
            @RequestParam(required = false) Status status) { // Ejemplo:
/partidas?status=WAITING

        List<Partida> partidas;
        if (status != null) {
            partidas = partidaService.findByStatus(status);
        } else {
            partidas = partidaService.findAll();
        }
        return new ResponseEntity<>(partidas, HttpStatus.OK); // 200 OK
    }

    // 2. GET BY ID (Un solo elemento)
    @GetMapping("/{id}")
    public ResponseEntity<Partida> getPartidaById(@PathVariable("id") Integer id)
{
        Partida p = partidaService.findById(id);
        if (p == null) {
            return new ResponseEntity<>(HttpStatus.NOT_FOUND); // 404
        }
        return new ResponseEntity<>(p, HttpStatus.OK); // 200 OK
    }

    // 3. POST (Crear)
    @PostMapping
    public ResponseEntity<?> createPartida(@Valid @RequestBody Partida
nuevaPartida) {
        // @Valid lanza 400 automáticamente si falla alguna anotación JPA/Jakarta
        try {
            User currentUser = userService.findCurrentUser();
            Partida partidaCreada = partidaService.create(nuevaPartida,
currentUser.getId());
            return new ResponseEntity<>(partidaCreada, HttpStatus.CREATED); // 201
CREATED
        } catch (BusinessException ex) {
            return new ResponseEntity<>(ex.getMessage(), HttpStatus.BAD_REQUEST);
// 400 BAD REQUEST
        }
    }

    // 4. PUT (Actualizar)
    @PutMapping("/{id}")
    public ResponseEntity<?> updatePartida(
            @PathVariable("id") Integer id,
            @Valid @RequestBody Partida datosActualizados) {
        try {
```

```
            Partida actualizada = partidaService.update(id, datosActualizados);
            return new ResponseEntity<>(actualizada, HttpStatus.OK); // 200 OK
        } catch (BusinessException ex) {
            return new ResponseEntity<>(ex.getMessage(), HttpStatus.BAD_REQUEST);
        }
    }

    // 5. DELETE (Borrar)
    @DeleteMapping("/{id}")
    public ResponseEntity<?> deletePartida(@PathVariable("id") Integer id) {
        try {
            User currentUser = userService.findCurrentUser();
            if (!currentUser.hasAuthority("ADMIN")) {
                return new ResponseEntity<>("Solo admins pueden borrar",
HttpStatus.FORBIDDEN); // 403
            }

            partidaService.delete(id);
            return new ResponseEntity<>("Partida borrada con éxito",
HttpStatus.OK); // 200 OK (o 204 NO_CONTENT)
        } catch (BusinessException ex) {
            return new ResponseEntity<>(ex.getMessage(), HttpStatus.BAD_REQUEST);
        }
    }
}
```

---

7. FRONTEND REACT (Componentes .js/.jsx)

Plantilla universal para un componente que hace `fetch` (traer datos), maneja estado de carga, errores y pinta los datos en una tabla.

JavaScript

```
import React, { useState, useEffect } from 'react';
import { Table, Button, Spinner } from 'reactstrap';

export default function MiListado() {
    // 1. ESTADOS
    const [datos, setDatos] = useState([]);
    const [loading, setLoading] = useState(true); // Controlar si está cargando
    const [error, setError] = useState(null);

    // 2. EFECTO AL MONTAR EL COMPONENTE
    useEffect(() => {
        const fetchData = async () => {
```

```
            try {
                // Si el proyecto usa un fetch personalizado con auth (ej:
authFetch), úsalo aquí
                const res = await fetch('/api/v1/partidas');
                if (!res.ok) {
                    throw new Error(`Error HTTP: ${res.status}`);
                }
                const data = await res.json();
                setDatos(data);
            } catch (err) {
                setError(err.message);
            } finally {
                setLoading(false); // Siempre quitamos el spinner al terminar
            }
        };

        fetchData();
    }, []); // Array vacío = se ejecuta solo 1 vez al cargar la página

    // 3. RENDERIZADOS CONDICIONALES
    if (loading) return <Spinner color="primary" />;
    if (error) return <h2 style={{ color: 'red' }}>Error: {error}</h2>;
    if (datos.length === 0) return <h1>NO HAY DATOS REGISTRADOS</h1>;

    // 4. RENDERIZADO PRINCIPAL (Tabla)
    return (
        <div className="container mt-4">
            <h1 className="text-center">Listado de Partidas</h1>
            <Table striped responsive>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Nombre</th>
                        <th>Fecha Inicio</th>
                        <th>Creador</th>
                        <th>Acciones</th>
                    </tr>
                </thead>
                <tbody>
                    {datos.map((item) => (
                        <tr key={item.id}>
                            <td>{item.id}</td>
                            <td>{item.name}</td>
                            {/* Manejo de fechas en JS */}
                            <td>{item.startDate ? new
Date(item.startDate).toLocaleDateString() : 'Sin fecha'}</td>
                            {/* CUIDADO con los nulos al acceder a sub-objetos */}
```

```
                            <td>{item.creator ? item.creator.username : 'Sin
asignar'}</td>
                            <td>
                                <Button color="info" size="sm">Ver</Button>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </Table>
        </div>
    );
}
```